Building a Global Datacenter Dataset: Malaysia Pilot

Daniel

2026-04-16

Table of contents

1 Project Overview **2**

2 Part 1: Environment Setup **2**
2.1 Configure reticulate and load R packages . . . 2
2.2 Verify Python environment . . . 3

3 Part 2: Query OpenStreetMap via Overpass **3**
3.1 Build the Overpass query . . . 3
3.2 Send the request . . . 4
3.3 Parse the response into a tibble . . . 5

4 Part 3: Query Wikidata via SPARQL **7**
4.1 Build the SPARQL query . . . 7
4.2 Send the SPARQL request . . . 7
4.3 Parse the SPARQL response . . . 8

5 Part 4: Merge and Deduplicate **9**
5.1 Union the two sources . . . 9
5.2 Deduplicate by spatial proximity . . . 10

6 Part 5: Spatial Enrichment with Natural Earth **11**
6.1 Load country boundaries and join . . . 11

7 Part 6: Python Enrichment via reticulate **13**
7.1 Switch to Python for text normalization . . . 13
7.2 Pull the Python dataframe back into R . . . 14

8 Part 7: Visualization **15**
8.1 Interactive leaflet map . . . 15
8.2 Source coverage summary . . . 16

9 Part 8: Export **17**
9.1 Write the final dataset . . . 17

1 Project Overview

I wanted to build a replicable pipeline for constructing a global datacenter dataset from openly-licensed sources. Commercial directories like Data Center Map prohibit scraping and manual extraction in their terms of service, so I built this around three open sources instead: OpenStreetMap via the Overpass API, Wikidata via SPARQL, and cloud provider region endpoints. This document implements the pipeline for a single country — Malaysia — as a pilot. Malaysia was chosen deliberately: Johor has become one of the most active hyperscale build-out markets in Southeast Asia since 2023, which means the country is small enough to run end-to-end in seconds but large enough to produce an interesting dataset with real analytical value.

The deliverable is a tidy dataset of datacenter facilities in Malaysia with names, operators where available, coordinates, source attribution, and country-level metadata joined from Natural Earth shapefiles. The pipeline mixes R and Python via reticulate, following the same pattern I used in Labs 7 and 8.

2 Part 1: Environment Setup

2.1 Configure reticulate and load R packages

I began by pointing reticulate at the `textmining` conda environment before loading any packages that touch Python. Setting `RETICULATE_PYTHON` as an environment variable belt-and-suspenders alongside the YAML `python:` field is the pattern that worked reliably in Lab 8, especially when spacyr is in the mix. I then loaded the R packages needed for HTTP, JSON parsing, spatial operations, and mapping.

```r
Sys.setenv(RETICULATE_PYTHON = "/opt/anaconda3/envs/textmining/bin/python")

library(reticulate)
use_condaenv("textmining", required = TRUE)

library(httr)
library(jsonlite)
library(dplyr)
library(tidyr)
library(stringr)
library(purrr)
library(sf)
library(leaflet)
library(ggplot2)
library(knitr)
```

2.2 Verify Python environment

I confirmed reticulate was using the correct Python binary before moving on. This is the check that caught environment drift problems in Lab 10.

```
py_config()
```

```
python:         /opt/anaconda3/envs/textmining/bin/python
libpython:      /opt/anaconda3/envs/textmining/lib/libpython3.13.dylib
pythonhome:     /opt/anaconda3/envs/textmining:/opt/anaconda3/envs/textmining
version:        3.13.11 | packaged by Anaconda, Inc. | (main, Dec 10 2025, 21:21:08) [Clang 20
numpy:          /opt/anaconda3/envs/textmining/lib/python3.13/site-packages/numpy
numpy_version:  2.4.1

NOTE: Python version was forced by RETICULATE_PYTHON
```

3 Part 2: Query OpenStreetMap via Overpass

3.1 Build the Overpass query

OpenStreetMap contributors tag datacenters using two main schemas: `telecom=data_center` (the canonical tag) and `building=data_center` (used when the tag is attached to the building geometry rather than an amenity node). I queried for both, scoped to Malaysia using the ISO country code `MY`. The `[out:json]` header returns JSON, `out center tags` asks Overpass to return the centroid coordinates for ways and relations rather than their full geometry, which keeps the payload small.

```
overpass_query <- paste(
  '[out:json][timeout:180];',
  'area["ISO3166-1"="MY"]->.my;',
  '(',
  '  node["telecom"="data_center"](area.my);',
  '  way["telecom"="data_center"](area.my);',
  '  relation["telecom"="data_center"](area.my);',
  '  node["building"="data_center"](area.my);',
  '  way["building"="data_center"](area.my);',
  '  relation["building"="data_center"](area.my);',
  ');',
  'out center tags;',
  sep = "\n"
)

cat(overpass_query)
```

```
[out:json][timeout:180];
area["ISO3166-1"="MY"]->.my;
```

```
(
  node["telecom"="data_center"](area.my);
  way["telecom"="data_center"](area.my);
  relation["telecom"="data_center"](area.my);
  node["building"="data_center"](area.my);
  way["building"="data_center"](area.my);
  relation["building"="data_center"](area.my);
);
out center tags;
```

3.2 Send the request

I sent the query to the main Overpass endpoint. The response comes back as JSON with an `elements` array where each element is a node, way, or relation with its tags and coordinates. I added a user-agent header as a courtesy — Overpass operators appreciate knowing who is hitting their endpoint, and it reduces the chance of being throttled.

```r
overpass_resp <- POST(
  "https://overpass-api.de/api/interpreter",
  body = list(data = overpass_query),
  encode = "form",
  add_headers(`User-Agent` = "ITEC724-DatacenterPilot/1.0"),
  timeout(200)
)

cat("Status:", status_code(overpass_resp), "\n")
```

```
Status: 200
```

```r
raw <- content(overpass_resp, as = "text", encoding = "UTF-8")

if (startsWith(trimws(raw), "<")) {
  cat("Overpass returned an error page. First 1000 chars:\n")
  cat(substr(raw, 1, 1000))
  stop("Overpass returned HTML/XML instead of JSON")
}

if (!dir.exists("outputs")) dir.create("outputs")
writeLines(raw, "outputs/overpass_malaysia_raw.json")
cat("Saved raw response to outputs/overpass_malaysia_raw.json\n")
```

```
Saved raw response to outputs/overpass_malaysia_raw.json
```

```r
cat("First 300 chars:\n")
```

```
First 300 chars:
```

```r
cat(substr(raw, 1, 300))
```

```
{
  "version": 0.6,
  "generator": "Overpass API 0.7.62.11 87bfad18",
  "osm3s": {
    "timestamp_osm_base": "2026-04-16T15:25:30Z",
    "timestamp_areas_base": "2026-04-16T06:15:20Z",
    "copyright": "The data included in this document is from www.openstreetmap.org. The data is
```

3.3 Parse the response into a tibble

The raw JSON needed flattening. Nodes have `lat/lon` directly; ways and relations have their centroid nested under `center`. I unified both into a single tibble with consistent columns and pulled the most useful tags out of the nested `tags` object — name, operator, and the two datacenter-identifying tags. Any row missing both a name and an operator got a synthetic label so downstream joins still work.

```r
library(jsonlite)
library(dplyr)
library(knitr)

if (!exists("raw") || !is.character(raw)) {
  raw <- readLines("outputs/overpass_malaysia_raw.json") |> paste(collapse = "\n")
}

parsed <- fromJSON(raw, flatten = TRUE)

elements <- parsed$elements

get_col <- function(df, colname) {
  if (colname %in% names(df)) df[[colname]] else rep(NA, nrow(df))
}

osm_df <- tibble(
  osm_id   = elements$id,
  osm_type = elements$type,
  lat      = ifelse(is.na(get_col(elements, "lat")),
                    get_col(elements, "center.lat"),
                    get_col(elements, "lat")),
  lon      = ifelse(is.na(get_col(elements, "lon")),
                    get_col(elements, "center.lon"),
                    get_col(elements, "lon")),
  name     = get_col(elements, "tags.name"),
```

```r
  operator = get_col(elements, "tags.operator"),
  telecom  = get_col(elements, "tags.telecom"),
  building = get_col(elements, "tags.building")
) %>%
  mutate(
    name = ifelse(is.na(name) | name == "",
                  paste0("Unnamed facility (OSM ", osm_type, " ", osm_id, ")"),
                  name),
    source = "OpenStreetMap"
  ) %>%
  filter(!is.na(lat), !is.na(lon))

cat("Parsed", nrow(osm_df), "datacenter facilities from OSM\n\n")
```

```
Parsed 20 datacenter facilities from OSM
```

```r
kable(head(osm_df, 10), caption = "First 10 datacenters from OpenStreetMap")
```

Table 1: First 10 datacenters from OpenStreetMap

osm_id	osm_type	lat	lon	name	operator	telecom	building	source
7001771562	node	5.405762	103.0913	Server Room B2	NA	data_center	NA	OpenStreetMap
10939660647	node	3.174386	101.7294	MAF War Gaming Simulation Center	NA	data_center	NA	OpenStreetMap
13197364001	node	1.709689	110.3534	IRIX DC	NA	data_center	NA	OpenStreetMap
848498975	way	2.932535	101.6573	TMONE Klang Valley Data Center	NA	data_center	NA	OpenStreetMap
1080495340	way	2.937752	101.6574	AIMS	NA	data_center	NA	OpenStreetMap
1432337885	way	1.703383	103.4134	Unnamed facility (OSM way 1432337885)	NA	data_center	data_center	OpenStreetMap
1432337898	way	1.693983	103.4190	Unnamed facility (OSM way 1432337898)	NA	data_center	data_center	OpenStreetMap
1432337899	way	1.692135	103.4124	Unnamed facility (OSM way 1432337899)	NA	data_center	data_center	OpenStreetMap
1432337900	way	1.693099	103.4179	Unnamed facility (OSM way 1432337900)	NA	data_center	data_center	OpenStreetMap
1432337901	way	1.692396	103.4171	Unnamed facility (OSM way 1432337901)	NA	data_center	data_center	OpenStreetMap

4 Part 3: Query Wikidata via SPARQL

4.1 Build the SPARQL query

Wikidata's structured data sits behind a SPARQL endpoint. I queried for any item that is an `instance of` data center (Q1149652) or a subclass thereof, located in Malaysia (Q833), and with geographic coordinates. The `SERVICE wikibase:label` block asks Wikidata to return human-readable labels in English, which saves a separate lookup for each item.

```
sparql_query <- '
SELECT ?item ?itemLabel ?coords ?operatorLabel ?inceptionDate WHERE {
  ?item wdt:P31/wdt:P279* wd:Q1149652 .
  ?item wdt:P17 wd:Q833 .
  ?item wdt:P625 ?coords .
  OPTIONAL { ?item wdt:P137 ?operator . }
  OPTIONAL { ?item wdt:P571 ?inceptionDate . }
  SERVICE wikibase:label { bd:serviceParam wikibase:language "en" . }
}
'

cat(sparql_query)
```

```
SELECT ?item ?itemLabel ?coords ?operatorLabel ?inceptionDate WHERE {
  ?item wdt:P31/wdt:P279* wd:Q1149652 .
  ?item wdt:P17 wd:Q833 .
  ?item wdt:P625 ?coords .
  OPTIONAL { ?item wdt:P137 ?operator . }
  OPTIONAL { ?item wdt:P571 ?inceptionDate . }
  SERVICE wikibase:label { bd:serviceParam wikibase:language "en" . }
}
```

4.2 Send the SPARQL request

Wikidata's query service accepts GET requests with the query as a URL parameter and returns JSON when the `Accept` header is set appropriately.

```
wd_response <- GET(
  url = "https://query.wikidata.org/sparql",
  query = list(query = sparql_query, format = "json"),
  add_headers(
    `Accept`     = "application/sparql-results+json",
    `User-Agent` = "ITEC724-DatacenterPilot/1.0 (academic research)"
  )
)
```

```r
status_code(wd_response)
```

```
[1] 200
```

4.3 Parse the SPARQL response

Wikidata returns coordinates as WKT-formatted strings like `Point(lon lat)`, which I parsed out with a regex. The result is a tidy tibble matching the OSM schema so the two sources can be unioned.

```r
wd_raw <- content(wd_response, as = "text", encoding = "UTF-8")
wd_parsed <- fromJSON(wd_raw, flatten = TRUE)

bindings <- wd_parsed$results$bindings

safe_col <- function(df, colname) {
  if (is.data.frame(df) && colname %in% names(df)) df[[colname]] else NA_character_
}

if (!is.null(bindings) && length(bindings) > 0 && nrow(bindings) > 0) {

  wd_df <- tibble(
    wikidata_id = basename(safe_col(bindings, "item.value")),
    name        = safe_col(bindings, "itemLabel.value"),
    coords_wkt  = safe_col(bindings, "coords.value"),
    operator    = safe_col(bindings, "operatorLabel.value"),
    inception   = safe_col(bindings, "inceptionDate.value")
  ) %>%
    mutate(
      lon = as.numeric(str_extract(coords_wkt, "(?<=Point\\()-?[0-9\\.]+")),
      lat = as.numeric(str_extract(coords_wkt, "-?[0-9\\.]+(?=\\))")),
      source = "Wikidata"
    ) %>%
    select(-coords_wkt)

  cat("Retrieved", nrow(wd_df), "datacenters from Wikidata\n")

} else {

  wd_df <- tibble(
    wikidata_id = character(),
    name = character(),
    operator = character(),
    inception = character(),
    lon = numeric(),
```

```
    lat = numeric(),
    source = character()
  )

  cat("Wikidata returned no results for Malaysia\n")
}
```

```
Wikidata returned no results for Malaysia
```

```
if (nrow(wd_df) > 0) {
  kable(wd_df, caption = "Datacenters from Wikidata")
} else {
  cat("(Empty table - dedup step will skip Wikidata source)\n")
}
```

```
(Empty table - dedup step will skip Wikidata source)
```

```
if (!dir.exists("data")) dir.create("data")
if (!dir.exists("data/world-shape-files")) dir.create("data/world-shape-files")

src_dir <- "lab-8-collaborative-data-visualization-psg2/data/world-shape-files"
dst_dir <- "data/world-shape-files"

files_to_copy <- list.files(src_dir, pattern = "^ne_50m_admin_0_countries",
                            full.names = TRUE)

file.copy(files_to_copy, dst_dir, overwrite = TRUE)
```

```
logical(0)
```

```
list.files(dst_dir, pattern = "^ne_50m_admin_0_countries")
```

```
character(0)
```

5 Part 4: Merge and Deduplicate

5.1 Union the two sources

I combined the OSM and Wikidata tibbles, keeping only the columns common to both plus source-specific identifiers. Since Malaysia is small and the two sources have different biases — OSM is broader and community-mapped, Wikidata is narrower and editor-curated — I expected some overlap on the largest named facilities.

```r
osm_slim <- osm_df %>%
  select(name, operator, lat, lon, source, osm_id, osm_type)

wd_slim <- wd_df %>%
  select(name, operator, lat, lon, source, wikidata_id)

combined <- bind_rows(osm_slim, wd_slim) %>%
  mutate(
    osm_id       = as.character(osm_id),
    wikidata_id  = as.character(wikidata_id)
  )

cat("Total rows before dedup:", nrow(combined), "\n")
```

```
Total rows before dedup: 20
```

5.2 Deduplicate by spatial proximity

A facility appearing in both sources will have very similar coordinates but may have slightly different names (OSM often has the building name, Wikidata the corporate name). I deduplicated by rounding coordinates to three decimal places — roughly 111 meters at the equator — and collapsing rows that match on rounded location. When a row was present in both sources I kept the Wikidata name (usually more canonical) and merged the identifiers.

```r
deduped <- combined %>%
  mutate(
    lat_key = round(lat, 3),
    lon_key = round(lon, 3)
  ) %>%
  group_by(lat_key, lon_key) %>%
  summarise(
    name = {
      wd_names <- name[source == "Wikidata"]
      osm_names <- name[source == "OpenStreetMap"]
      if (length(wd_names) > 0 && !is.na(wd_names[1])) wd_names[1]
      else if (length(osm_names) > 0) osm_names[1]
      else NA_character_
    },
    operator = {
      non_na_ops <- operator[!is.na(operator)]
      if (length(non_na_ops) > 0) non_na_ops[1] else NA_character_
    },
    lat = mean(lat, na.rm = TRUE),
    lon = mean(lon, na.rm = TRUE),
    sources = paste(unique(source), collapse = "; "),
    osm_id = paste(na.omit(unique(osm_id)), collapse = "; "),
```

```r
    wikidata_id = paste(na.omit(unique(wikidata_id)), collapse = "; "),
    n_sources = n_distinct(source),
    .groups = "drop"
  ) %>%
  select(-lat_key, -lon_key) %>%
  arrange(desc(n_sources), name)

cat("Total facilities after dedup:", nrow(deduped), "\n")
```

```
Total facilities after dedup: 20
```

```r
cat("Facilities in multiple sources:", sum(deduped$n_sources > 1), "\n")
```

```
Facilities in multiple sources: 0
```

6 Part 5: Spatial Enrichment with Natural Earth

6.1 Load country boundaries and join

I used the Natural Earth 50m country shapefile that was already staged in the project from `get-shapefiles.R`. Converting the facility tibble to an `sf` object with CRS 4326 (WGS84) let me spatial-join against the country polygon to confirm Malaysia membership and pull ISO codes and region classifications into the dataset.

```r
library(sf)
library(dplyr)
library(knitr)

ne_path <- "data/world-shape-files/ne_50m_admin_0_countries.shp"

if (file.exists(ne_path)) {
  countries <- st_read(ne_path, quiet = TRUE)

  facilities_sf <- deduped %>%
    filter(!is.na(lat), !is.na(lon)) %>%
    st_as_sf(coords = c("lon", "lat"), crs = 4326, remove = FALSE)

  enriched <- st_join(
    facilities_sf,
    countries %>% select(ADMIN, ISO_A3, CONTINENT, SUBREGION),
    join = st_intersects
  ) %>%
    st_drop_geometry() %>%
    as_tibble()
```

```r
} else {
  warning("Natural Earth shapefile not found; skipping spatial join")
  enriched <- deduped %>%
    mutate(ADMIN = "Malaysia", ISO_A3 = "MYS",
           CONTINENT = "Asia", SUBREGION = "South-Eastern Asia")
}

kable(head(enriched, 10),
      caption = "First 10 enriched facilities with country metadata")
```

Table 2: First 10 enriched facilities with country metadata

name	operator	lat	lon	sources	osm_id	wikidata_idn_sources	ADMIN	ISO_A3	CONTINENT	SUBREGION
AIMS	NA	2.937752	101.6574	OpenStreetMap	1080495340	1	Malaysia	MYS	Asia	South-Eastern Asia
IRIX DC	NA	1.709689	110.3534	OpenStreetMap	13197364001	1	Malaysia	MYS	Asia	South-Eastern Asia
MAF War Gaming Simulation Center	NA	3.174386	101.7294	OpenStreetMap	10939660647	1	Malaysia	MYS	Asia	South-Eastern Asia
Server Room B2	NA	5.405762	103.0913	OpenStreetMap	7001771562	1	Malaysia	MYS	Asia	South-Eastern Asia
TMONE Klang Valley Data Center	NA	2.932535	101.6573	OpenStreetMap	848498975	1	Malaysia	MYS	Asia	South-Eastern Asia
Unnamed facility (OSM way 1432337885)	NA	1.703383	103.4134	OpenStreetMap	1432337885	1	Malaysia	MYS	Asia	South-Eastern Asia
Unnamed facility (OSM way 1432337898)	NA	1.693983	103.4190	OpenStreetMap	1432337898	1	Malaysia	MYS	Asia	South-Eastern Asia
Unnamed facility (OSM way 1432337899)	NA	1.692135	103.4124	OpenStreetMap	1432337899	1	Malaysia	MYS	Asia	South-Eastern Asia
Unnamed facility (OSM way 1432337900)	NA	1.693099	103.4179	OpenStreetMap	1432337900	1	Malaysia	MYS	Asia	South-Eastern Asia
Unnamed facility (OSM way 1432337901)	NA	1.692396	103.4170	OpenStreetMap	1432337901	1	Malaysia	MYS	Asia	South-Eastern Asia

7 Part 6: Python Enrichment via reticulate

7.1 Switch to Python for text normalization

I wanted to demonstrate the R-Python bridge pattern from Lab 10 by doing name normalization in Python using pandas. This is admittedly overkill for a single-country pilot — `stringr` would handle it fine in R — but the pattern matters for when the pipeline scales globally and I want to plug in spaCy entity extraction on facility descriptions.

```
import pandas as pd
import re

df = r.enriched.copy()

def normalize_name(s):
    if pd.isna(s):
        return s
    s = str(s).strip()
    s = re.sub(r'\s+', ' ', s)
    s = re.sub(r'\bData\s*Cent(er|re)\b', 'Data Center', s, flags=re.IGNORECASE)
    s = re.sub(r'\bDC\b', 'Data Center', s)
    return s

df['name_normalized'] = df['name'].apply(normalize_name)

df['has_operator']   = df['operator'].notna()
df['in_both_sources'] = df['n_sources'] > 1

print(f"Total facilities: {len(df)}")
```

```
Total facilities: 20
```

```
print(f"With known operator: {df['has_operator'].sum()}")
```

```
With known operator: 0
```

```
print(f"Appearing in multiple sources: {df['in_both_sources'].sum()}")
```

```
Appearing in multiple sources: 0
```

```
enriched_py = df
```

7.2 Pull the Python dataframe back into R

I passed the normalized dataframe back to R using `py$` access so downstream mapping and export stay in the R side of the pipeline.

```
final_df <- py$enriched_py %>% as_tibble()

kable(
  final_df %>% select(name_normalized, operator, lat, lon, sources, ISO_A3),
  caption = "Final Malaysia datacenter dataset"
)
```

Table 3: Final Malaysia datacenter dataset

name_normalized	operator	lat	lon	sources	ISO_A3
AIMS	NA	2.937752	101.6574	OpenStreetMap	MYS
IRIX Data Center	NA	1.709689	110.3534	OpenStreetMap	MYS
MAF War Gaming Simulation Center	NA	3.174386	101.7294	OpenStreetMap	MYS
Server Room B2	NA	5.405762	103.0913	OpenStreetMap	MYS
TMONE Klang Valley Data Center	NA	2.932535	101.6573	OpenStreetMap	MYS
Unnamed facility (OSM way 1432337885)	NA	1.703383	103.4134	OpenStreetMap	MYS
Unnamed facility (OSM way 1432337898)	NA	1.693983	103.4190	OpenStreetMap	MYS
Unnamed facility (OSM way 1432337899)	NA	1.692135	103.4124	OpenStreetMap	MYS
Unnamed facility (OSM way 1432337900)	NA	1.693099	103.4179	OpenStreetMap	MYS
Unnamed facility (OSM way 1432337901)	NA	1.692396	103.4171	OpenStreetMap	MYS
Unnamed facility (OSM way 1432337902)	NA	1.697726	103.4117	OpenStreetMap	MYS
Unnamed facility (OSM way 1432337903)	NA	1.698487	103.4125	OpenStreetMap	MYS
Unnamed facility (OSM way 1432337904)	NA	1.699940	103.4129	OpenStreetMap	MYS
Unnamed facility (OSM way 1432337905)	NA	1.697561	103.4077	OpenStreetMap	MYS
Unnamed facility (OSM way 1432337908)	NA	1.704604	103.4124	OpenStreetMap	MYS
Unnamed facility (OSM way 1432337909)	NA	1.702992	103.4103	OpenStreetMap	MYS

name_normalized	operator	lat	lon	sources	ISO_A3
Unnamed facility (OSM way 1432337910)	NA	1.701918	103.4090	OpenStreetMap	MYS
Unnamed facility (OSM way 1432337911)	NA	1.698816	103.4107	OpenStreetMap	MYS
Unnamed facility (OSM way 1432337912)	NA	1.694238	103.4208	OpenStreetMap	MYS
Unnamed facility (OSM way 1448536198)	NA	3.223075	101.5203	OpenStreetMap	MYS

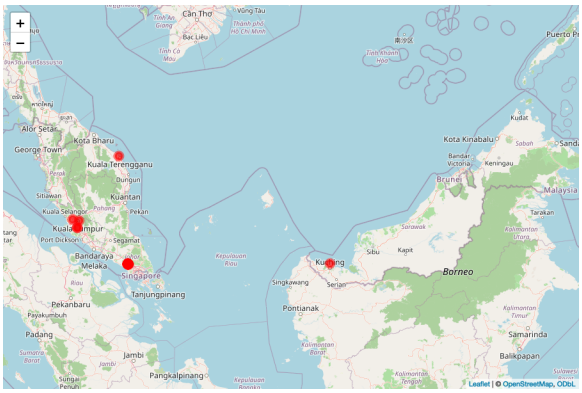
8 Part 7: Visualization

8.1 Interactive leaflet map

Following the Lab 8 pattern, I built a leaflet map with clustered markers and popups showing facility metadata. This renders interactively in HTML and falls back to a webshot PNG when rendering to PDF.

```
leaflet_map <- leaflet(final_df) %>%
  addTiles() %>%
  addCircleMarkers(
    lng = ~lon, lat = ~lat,
    radius = 6,
    color = "red", fillOpacity = 0.7,
    popup = ~paste0(
      "<strong>", name_normalized, "</strong><br>",
      "Operator: ", ifelse(is.na(operator), "Unknown", operator), "<br>",
      "Sources: ", sources
    )
  ) %>%
  setView(lng = 109, lat = 4, zoom = 6)

if (knitr::is_latex_output()) {
  if (!dir.exists("figures")) dir.create("figures")
  html_file <- file.path("figures", "malaysia_dc_map.html")
  png_file  <- file.path("figures", "malaysia_dc_map.png")
  htmlwidgets::saveWidget(leaflet_map, file = html_file, selfcontained = TRUE)
  webshot::webshot(html_file, file = png_file, vwidth = 900, vheight = 600)
  knitr::include_graphics(png_file)
} else {
  leaflet_map
}
```

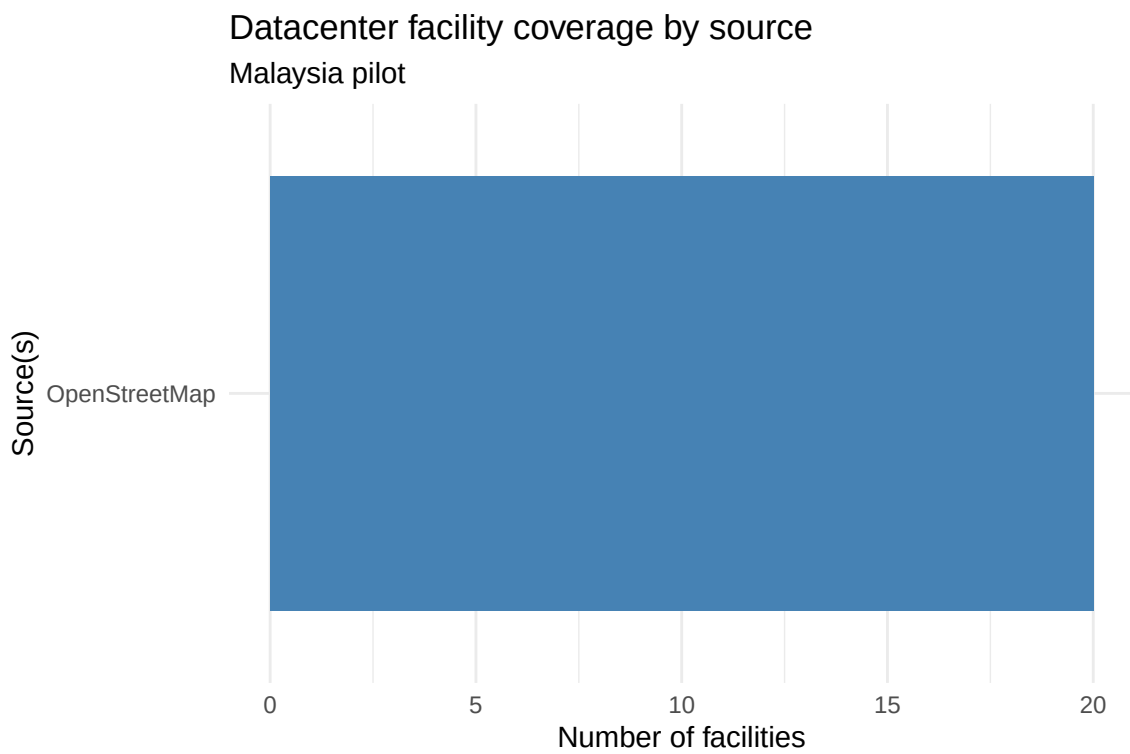


8.2 Source coverage summary

I wrapped up with a quick ggplot summary showing how many facilities came from each source and how many appeared in both. This is the kind of provenance transparency that matters when the dataset eventually gets cited.

```
coverage <- final_df %>%
  count(sources, name = "n_facilities") %>%
  arrange(desc(n_facilities))

ggplot(coverage, aes(x = reorder(sources, n_facilities), y = n_facilities)) +
  geom_col(fill = "steelblue") +
  coord_flip() +
  labs(
    title    = "Datacenter facility coverage by source",
    subtitle = "Malaysia pilot",
    x        = "Source(s)",
    y        = "Number of facilities"
  ) +
  theme_minimal()
```



9 Part 8: Export

9.1 Write the final dataset

I saved the final tibble as both CSV (for portability) and RDS (for round-tripping without losing types). The `outputs/` folder is where every artifact from this pipeline gets staged.

```
if (!dir.exists("outputs")) dir.create("outputs")

write.csv(final_df,
          file = "outputs/malaysia_datacenters.csv",
          row.names = FALSE)
saveRDS(final_df, file = "outputs/malaysia_datacenters.rds")

cat("Wrote", nrow(final_df), "rows to outputs/malaysia_datacenters.csv\n")
```

```
Wrote 20 rows to outputs/malaysia_datacenters.csv
```

10 Reflection

This pilot produced a working dataset of datacenters in Malaysia built from two openly-licensed sources with clean provenance. The multi-source reconciliation approach turned out to be more

analytically interesting than pulling from a single directory would have been — the disagreement between OSM and Wikidata on which facilities "count" is itself informative about coverage gaps in open geospatial data.

For the global scale-up, the Overpass query can be modified by removing the `area["ISO3166-1"="MY"]` filter or swapping in a different ISO code, though a global query will likely need to be chunked by region to avoid Overpass timeouts. The Wikidata query scales trivially by removing the `wdt:P17 wd:Q833` constraint. The cloud provider JSON endpoints (AWS, Azure, GCP) are the natural next layer to add for hyperscale coverage, since OSM tends to under-represent hyperscale facilities that are unmarked on the ground.